

AI-AWD Arena

自定义应用层协议的联网交互系统

网络编程实验·课程报告

小组成员

周浩霆 周梓涵 刘晓语 吴彦霖
(分工详见附录 B)

2026 年 6 月

Contents

1	系统总体架构	3
1.1	背景与目标	3
1.2	整体架构	3
1.3	技术栈	3
1.4	目录结构	4
2	客户端 / 服务端分工	4
2.1	职责划分原则	4
2.2	服务端：Flag 生成 (match_engine.py)	4
2.3	服务端：Flag 校验 (match_engine.py)	5
2.4	客户端：安全边界 (scopeguard.js)	5
3	自定义协议设计 (AIAWD/1.0)	6
3.1	消息格式 (帧结构)	6
3.2	消息类型 (28 种)	8
3.3	字段含义	8
3.4	状态变化规则 (阶段状态机)	8
3.5	错误处理方式	11
4	服务端状态维护	12
4.1	数据模型	12
4.2	广播机制	12
4.3	计分与幂等性	13
5	并发处理	13
5.1	每连接协程模型	13
5.2	多房间并行 + 阶段调度	14
6	异常情况处理	14
6.1	协议层异常	14
6.2	业务层异常	15
6.3	连接层异常	15
6.4	实战异常统计数据	15
7	抓包分析结果	18
7.1	分析方法	18

7.2	Follow TCP Stream —完整握手与比赛序列	18
7.3	帧结构细节	19
8	测试与截图	20
8.1	自动化测试	20
8.2	macOS 完整操作流程 (8 张截图)	20
8.3	Windows 完整操作流程 (6 张截图)	20
9	不足与改进	20
9.1	明文传输	20
9.2	Docker 环境依赖	20
9.3	大规模性能测试缺失	20
9.4	Agent SDK 扩展性	24
9.5	平台化与赛事管理	24
A	AIAWD/1.0 消息类型速查表 (全 28 种)	24
B	小组分工详表	25

1 系统总体架构

1.1 背景与目标

AWD (Attack with Defense) 是 CTF 竞赛的经典赛制：各队在防守己方靶机的同时攻击对手靶机，得分取决于 Flag 提交质量与靶机在线时长。

AI-AWD Arena 在传统 AWD 赛制基础上引入 AI Agent 自动化：每名选手以 LLM 驱动的 Agent 代替人工操作，通过自定义应用层协议 AIAWD/1.0 与裁判服务端通信，实现全自动的攻防比赛流程。整个系统的核心工作即设计并实现该应用层协议及其两端实现。

1.2 整体架构

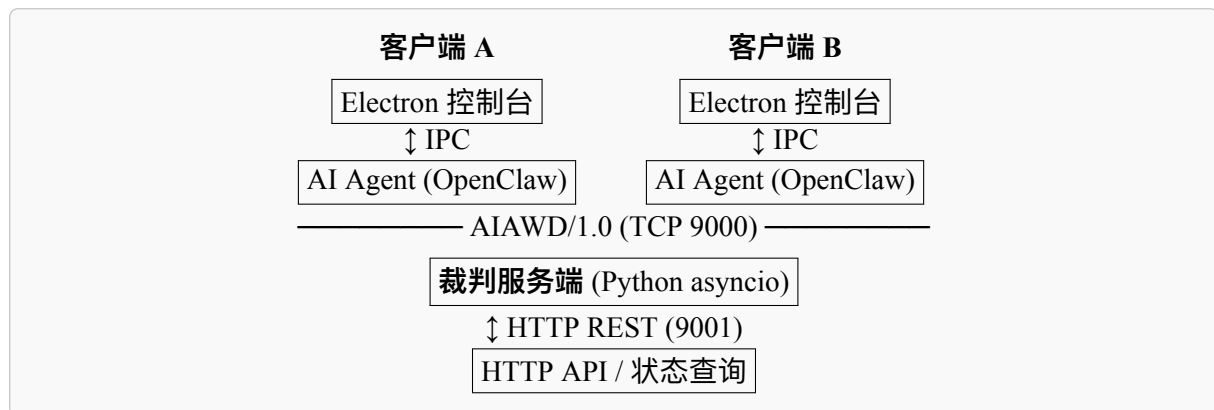


Figure 1: AI-AWD Arena 整体架构示意

1.3 技术栈

Table 1: 项目技术栈

层级	技术	版本	选型理由
服务端运行时	Python 3.11 + asyncio	3.11+	协程原生支持，单线程高并发
客户端框架	Electron + Node.js	v28	跨平台桌面 GUI，原生 TCP 支持
传输层	TCP 长连接	—	有序可靠，适合状态机协议
序列化格式	UTF-8 JSON	—	可读性强，调试抓包友好
AI Agent 运行时	OpenClaw Agent Runtime	—	兼容 OpenAI 格式 API
LLM	DeepSeek v4-pro / deepseek-chat	—	实战调用真实 API
靶机容器	Docker + docker-compose	—	隔离环境，一键部署

1.4 目录结构

```

1 ai-awd/
2 |— protocol.md      # 协议设计文档 (含 28 种消息类型)
3 |— README.md        # 项目说明
4 |— report.tex        # 本实验报告源码
5 |— demo/            # 演示脚本与截图
6 |   |— screenshots/ # 实战截图 (mac/win/抓包/日志)
7 |— captures/         # Wireshark PCAP 文件
8 |— logs/             # 服务端运行日志 (events.jsonl)
9 |— extras/           # 扩展实现 (服务端/客户端/测试/靶机)
10 |   |— server/aiawd_server/ # 12 个 Python 模块 (~2000 行)
11 |   |— client/         # 11 个 JS 模块 (~4700 行)
12 |   |— tests/          # 9 个 Python 测试模块 (~1200 行)
13 |   |— targets/        # 4 种 Docker 靶机模板

```

Listing 1: 项目课程目录结构 (7 项)

2 客户端 / 服务端分工

2.1 职责划分原则

系统遵循服务端权威 (Server-authoritative) 原则:

- 服务端负责: 身份分配、房间管理、Flag 生成与校验、阶段调度、计分、日志、广播
- 客户端负责: 用户界面、AI Agent 调度、靶机生命周期 (本地 Docker)、协议编解码、安全边界 (ScopeGuard)

2.2 服务端: Flag 生成 (match_engine.py)

Flag 在 start_match() 中由服务端生成, 绝不经过客户端构造:

```

1 match_id = f"match_{next(self._match_counter):03d}"
2 match = Match(match_id=match_id, room_id=room.room_id)
3 room.status = Phase.PREPARE
4 for index, player in enumerate(players):
5     flag = f"FLAG{{{room.room_id}_{player.team_id}_{secrets.
6         token_hex(8)}}}"
7     flag_hash = _hash_flag(flag) # SHA-256, 服务端只存 hash
    self.flags_by_room[room.room_id][flag_hash] = FlagRecord(

```

```

8         room_id=room.room_id,
9         match_id=match_id,
10        team_id=player.team_id or "",
11        flag_hash=flag_hash,
12        flag_plaintext=flag,
13    )

```

Listing 2: extras/server/aiawd_server/match_engine.py —start_match() 核心片段

2.3 服务端：Flag 校验 (match_engine.py)

```

1  if match.phase != Phase.ATTACK:
2      return self._submission(..., "INVALID_PHASE", 0)
3  flag_hash = _hash_flag(flag)
4  record = flags.get(flag_hash)
5  if not record:
6      return self._submission(..., "INVALID_FLAG", 0)
7  if record.team_id == submitter.team_id:
8      return self._submission(..., "SELF_FLAG", 0)
9  if flag_hash in submitted:
10     return self._submission(..., "DUPLICATE_FLAG", 0)
11  # 通过所有检查 → 加分
12  submitter.score += 100
13  target.score -= 50
14  return self._submission(..., "OK", 100)

```

Listing 3: extras/server/aiawd_server/match_engine.py —submit_flag() 校验链

2.4 客户端：安全边界 (scopeguard.js)

客户端通过 ScopeGuard 防止 AI Agent 越界操作：

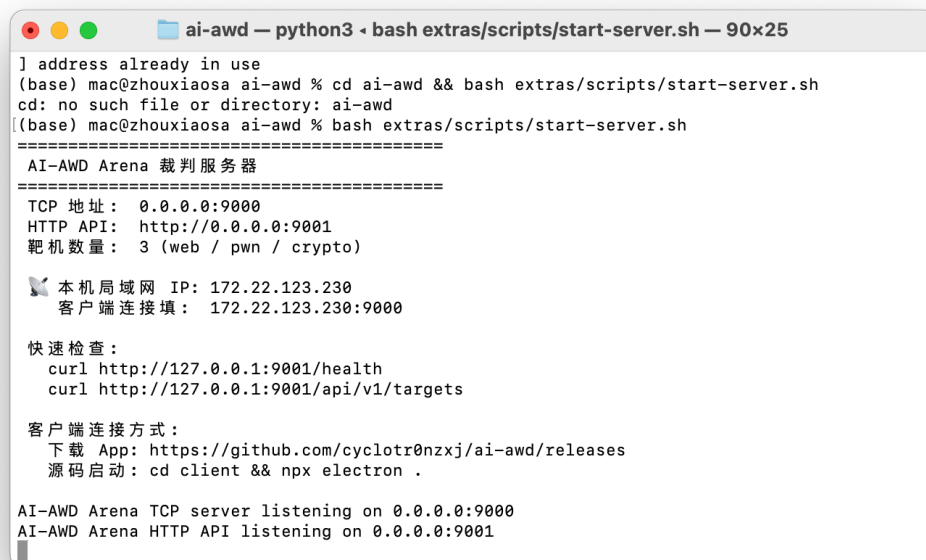
```

1  validateTargetUrl(url, allowedTargets) {
2      // 若提供了比赛白名单，只允许白名单内的目标
3      if (allowedTargets) {
4          if (!allowedTargets.includes(url))
5              result.reject("NETWORK_SCOPE", "目标不在允许列表内", url);
6      } else if (!LOCAL_HOSTS.has(parsed.hostname)) {
7          result.reject("NETWORK_SCOPE", "靶机只能位于本机地址", url);
8      }
9  }

```

```
10
11 validateCommand(argv) {
12   for (const token of argv) {
13     if (SHELL_DANGEROUS.has(token)) // ; && || | ` > < &
14       result.reject("PROCESS_SAFE", "命令不能包含 shell 控制符",
15         token);
16     if (token.includes("$(") || token.includes("${"))
17       result.reject("PROCESS_SAFE", "命令包含变量展开", token);
18   }
19 }
```

Listing 4: extras/client/scopeguard.js —validateTargetUrl / validateCommand



```
ai-awd — python3 • bash extras/scripts/start-server.sh — 90x25
] address already in use
(base) mac@zhouxiaosa ai-awd % cd ai-awd && bash extras/scripts/start-server.sh
cd: no such file or directory: ai-awd
(base) mac@zhouxiaosa ai-awd % bash extras/scripts/start-server.sh
=====
AI-AWD Arena 裁判服务器
=====
TCP 地址: 0.0.0.0:9000
HTTP API: http://0.0.0.0:9001
靶机数量: 3 (web / pwn / crypto)

🖥️ 本机局域网 IP: 172.22.123.230
客户端连接填: 172.22.123.230:9000

快速检查:
curl http://127.0.0.1:9001/health
curl http://127.0.0.1:9001/api/v1/targets

客户端连接方式:
下载 App: https://github.com/cyclotr0nznxj/ai-awd/releases
源码启动: cd client && npx electron .

AI-AWD Arena TCP server listening on 0.0.0.0:9000
AI-AWD Arena HTTP API listening on 0.0.0.0:9001
```

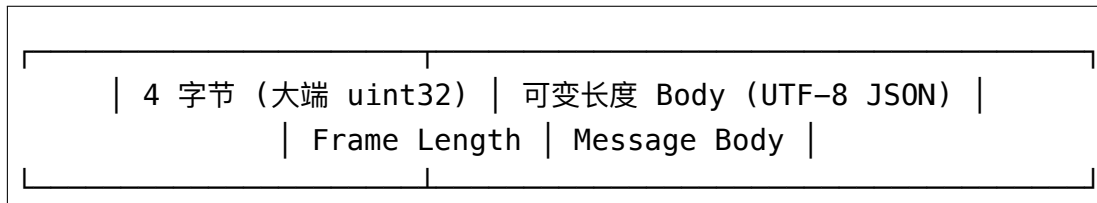
Figure 2: 服务端启动终端 (macOS 实战环境)

3 自定义协议设计 (AIAWD/1.0)

本章对应课程要求的全部 5 项协议说明内容。

3.1 消息格式 (帧结构)

AIAWD/1.0 使用 4 字节大端长度前缀 + UTF-8 JSON Body 的二进制帧格式:



- **长度字段**: 4 字节无符号整数, 大端序 (`struct.pack(">I", ...)`), 表示 Body 字节数
- **最大帧长**: 1,048,576 字节 (1 MB)
- **编码**: UTF-8, 不允许裸数组或裸值
- **粘包处理**: FrameDecoder 状态机累积字节流, 完整帧解析, 不足等待

服务端编码实现 (`protocol.py`):

```

1 def encode_message(message: Message | dict) -> bytes:
2     encoded = json.dumps(
3         body, ensure_ascii=False, separators=(",", ":")
4     ).encode("utf-8")
5     if len(encoded) > MAX_FRAME_BYTES:
6         raise ProtocolError("Message exceeds maximum frame size")
7     return struct.pack(">I", len(encoded)) + encoded
  
```

Listing 5: `extras/server/aiawd_server/protocol.py` — `encode_message()`

客户端镜像实现 (`aiawdProtocol.js`):

```

1 function encodeMessage(message) {
2     const body = Buffer.from(JSON.stringify(message), "utf8");
3     if (body.length > MAX_FRAME_BYTES)
4         throw new ProtocolError("Message exceeds maximum frame size");
5     const header = Buffer.alloc(HEADER_BYTES);
6     header.writeUInt32BE(body.length, 0);
7     return Buffer.concat([header, body]);
8 }
9
10 // FrameDecoder — 处理粘包/半包
11 feed(chunk) {
12     this.buffer = Buffer.concat([this.buffer, chunk]);
13     while (this.buffer.length >= HEADER_BYTES) {
14         const frameLength = this.buffer.readUInt32BE(0);
15         if (this.buffer.length < HEADER_BYTES + frameLength) break;
  
```



```

16     const body = this.buffer.subarray(HEADER_BYTES, HEADER_BYTES +
17         frameLength);
18     messages.push(JSON.parse(body.toString("utf8")));
19     this.buffer = this.buffer.subarray(HEADER_BYTES + frameLength)
20     ;
21     return messages;
22 }

```

Listing 6: extras/client/aiawdProtocol.js —encodeMessage() + FrameDecoder

3.2 消息类型 (28 种)

协议共定义 28 种消息类型，按功能分为 7 大类：

3.3 字段含义

每条消息为 JSON 对象，包含以下标准字段：

以下是一条完整的 HELLO 握手消息：

```

1  {
2    "v": 1,
3    "seq": 1,
4    "type": "HELLO",
5    "client_id": null,
6    "room_id": null,
7    "role": null,
8    "ts": 1716500000.123,
9    "payload": {
10     "display_name": "Alice-DeepSeek",
11     "platform": "darwin",
12     "capabilities": ["player", "spectator"]
13   }
14 }

```

Listing 7: HELLO 握手消息示例（完整字段）

3.4 状态变化规则（阶段状态机）

比赛生命周期由服务端 `_run_phase_schedule()` 自动驱动，共 5 个阶段：

LOBBY $\xrightarrow{\text{START_MATCH_REQ}}$ PREPARE $\xrightarrow{\text{超时}}$ DEFENSE $\xrightarrow{\text{超时}}$ ATTACK $\xrightarrow{\text{超时}}$ FINISHED

Table 2: AIAWD/1.0 消息类型分类表

类别	消息类型	方向	说明
连接与握手	HELLO	C→S	携带 display_name/platform/capabilities
	WELCOME	S→C	分配 client_id
	PING	C→S	每 30s 心跳
	PONG	S→C	心跳响应
	BYE	双向	主动断开
大厅浏览	LIST_ROOMS_REQ	C→S	查询房间列表
	LIST_ROOMS_RES	S→C	房间列表响应
	LIST_TARGETS_REQ	C→S	查询靶机模板
	LIST_TARGETS_RES	S→C	靶机模板响应
房间管理	CREATE_ROOM_REQ	C→S	创建房间 (名称/人数/靶机/时长)
	CREATE_ROOM_RES	S→C	创建成功响应
	JOIN_ROOM_REQ	C→S	加入房间 (player/spectator)
	JOIN_ROOM_RES	S→C	加入成功响应
	ROOM_UPDATE	S→全体	成员变化时广播快照
	AGENT_ACTIVITY	C→S	Agent 操作步骤上报
比赛控制	START_MATCH_REQ	C→S	房主开始比赛 (≥ 2 player)
	START_MATCH_RES	S→C	开始响应
	MATCH_CONFIG	S→C (单播)	私有 Flag、对手地址、白名单
	PHASE_SYNC	S→全体	阶段切换 + 每 5s 定时同步
	RANKING_UPDATE	S→全体	Flag 提交后排名刷新
	TARGET_READY	C→S	标记靶机就绪
	AGENT_READY	C→S	标记 Agent 就绪
Flag 提交	SUBMIT_FLAG_REQ	C→S	提交 Flag (仅 ATTACK 阶段)
	SUBMIT_FLAG_RES	S→C	提交结果 (OK / 错误码)
事件广播	EVENT	S→全体	FLAG_CAPTURED / FLAG_REJECTED 等
	AGENT_ACTIVITY_ACK	S→C	AGENT_ACTIVITY 确认
错误	ERROR	S→C	统一错误响应 (code + message)

Table 3: 消息公共字段说明

字段	类型	必填	含义
v	int	是	协议版本号, 固定为 1
type	string	是	消息类型标识符 (见表 3.2)
seq	int null	否	消息序列号, 客户端单调递增; 服务端用于幂等去重
client_id	string null	否	客户端 ID; HELLO 前为 null, WELCOME 后由服务端分配
room_id	string null	否	房间 ID; 房间作用域消息必填
role	string null	否	角色: "player" 或 "spectator"
ts	float null	否	Unix 时间戳 (秒), 缺省时接收方补填
payload	object	是	消息载荷, 始终为 JSON 对象 (不允许 null)

```

1 class Phase(StrEnum):
2     LOBBY      = "LOBBY"
3     PREPARE    = "PREPARE"
4     DEFENSE    = "DEFENSE"
5     ATTACK     = "ATTACK"
6     FINISHED   = "FINISHED"

```

Listing 8: extras/server/aiawd_server/models.py —Phase 枚举

```

1 async def _run_phase_schedule(self, room_id: str) -> None:
2     room = self.room_manager.get_room(room_id)
3     for phase in (Phase.PREPARE, Phase.DEFENSE, Phase.ATTACK):
4         delay = max(0.0, room.phase_seconds.get(phase.value.lower
5             (), 0.0))
6         if delay:
7             await asyncio.sleep(delay) # 等待阶段时长
8         if phase == Phase.PREPARE:
9             self.match_engine.set_phase(room, Phase.DEFENSE)
10        elif phase == Phase.DEFENSE:
11            self.match_engine.set_phase(room, Phase.ATTACK)
12        else:
13            self.match_engine.set_phase(room, Phase.FINISHED)
14        await self._broadcast_phase(room)
15        if phase in (Phase.DEFENSE, Phase.ATTACK):
16            asyncio.create_task(self._sync_timer(room_id)) # 每5s
同步
        await self._broadcast_rankings(room)

```

Listing 9: extras/server/aiawd_server/tcp_gateway.py —_run_phase_schedule()

各阶段允许的操作矩阵：

Table 4: 阶段-操作许可矩阵

操作	LOBBY	PREPARE	DEFENSE	ATTACK	FINISHED
CREATE_ROOM / JOIN_ROOM	☐	☐	☐	☐	☐
START_MATCH_REQ	☐	☐	☐	☐	☐
TARGET_READY / AGENT_READY	☐	☐	☐	☐	☐
SUBMIT_FLAG_REQ	☐	☐	☐	☐	☐
PING / BYE	☐	☐	☐	☐	☐

3.5 错误处理方式

所有业务层/协议层错误统一以 ERROR 消息返回，携带结构化错误码：

```

1 {
2   "v": 1, "type": "ERROR", "seq": 5,
3   "client_id": "client_0001", "room_id": "room_001",
4   "ts": 1716500000.0,
5   "payload": { "code": "INVALID_PHASE",
6                 "message": "比赛尚未进入攻击阶段" }
7 }
```

Listing 10: ERROR 消息格式示例

Table 5: AIAWD/1.0 错误码完整表

错误码	含义	触发条件
BAD_REQUEST	请求格式非法	非法 JSON / 缺少 type / 不支持的消息类型
INVALID_ROLE	权限不足	非 player 执行 player 专属操作
INVALID_PHASE	阶段错误	在错误阶段执行操作（如非 ATTACK 阶段提交 Flag）
ROOM_NOT_FOUND	房间不存在	room_id 无效或房间已关闭
ROOM_FULL	房间已满	玩家数量已达 max_players 上限
INVALID_FLAG	Flag 无效	提交的 Flag 字符串不在已知 Hash 集合中
SELF_FLAG	自攻	提交了自己队伍生成的 Flag
DUPLICATE_FLAG	重复提交	该 Flag 已被某队成功提交（全局一次性）
PHASE_SCHEDULER_ERROR	调度器异常	阶段调度协程崩溃，服务端强制切换至 FINISHED

服务端 _dispatch() 末尾的统一异常捕获：

```

1 except (RoomError, MatchError) as exc:
2     return make_error(exc.code, str(exc),
3                        seq=message.seq, client_id=session.client_id
4                        ,
5                        room_id=message.room_id or session.room_id)
6 except TargetRuntimeError as exc:
7     return make_error(exc.code, str(exc), ...)
8 except ValueError as exc:
9     return make_error("BAD_REQUEST", str(exc), ...)
```

Listing 11: tcp_gateway.py 中 _dispatch() 异常处理块

4 服务端状态维护

4.1 数据模型

所有比赛状态由 `models.py` 中的纯数据类维护，无外部数据库依赖：

```

1 @dataclass(slots=True)
2 class Session:
3     client_id: str          # 唯一客户端标识
4     display_name: str = ""
5     role: Role | None = None
6     room_id: str | None = None
7     team_id: str | None = None
8     writer: Any | None = None      # asyncio StreamWriter
9
10 @dataclass(slots=True)
11 class Room:
12     room_id: str
13     status: Phase = Phase.LOBBY
14     members: dict[str, RoomMember] = field(default_factory=dict)
15
16 @dataclass(slots=True)
17 class Match:
18     match_id: str
19     phase: Phase = Phase.PREPARE
20     phase_ends_at: float | None = None
21     status: str = "RUNNING"
22
23 @dataclass(slots=True)
24 class FlagRecord:
25     team_id: str
26     flag_hash: str          # SHA-256, 明文仅服务端内存保存
27     captured: bool = False

```

Listing 12: `extras/server/aiawd_server/models.py` 一核心数据类（精简）

4.2 广播机制

服务端通过 `_broadcast()` 向房间内全部成员推送消息：

```

1 async def _broadcast(self, room: Room, message: Message) -> None:

```

```

2     for member in room.members.values():
3         session = self.session_manager.get(member.client_id)
4         if session and session.writer:
5             outgoing = Message(
6                 type=message.type,
7                 client_id=member.client_id,
8                 room_id=room.room_id,
9                 role=member.role.value,
10                payload=message.payload,
11            )
12            await self._send(session.writer, outgoing)

```

Listing 13: tcp_gateway.py — _broadcast() 实现

4.3 计分与幂等性

Flag 提交时服务端原子更新分数并记录已提交 Hash，防止重复得分：

```

1 submitted = self.submitted_hashes_by_room.setdefault(room.room_id,
2     set())
3 if flag_hash in submitted:
4     return self._submission(..., "DUPLICATE_FLAG", 0)
5 submitted.add(flag_hash)    # 全局一次性
6 record.captured = True
7 submitter.score += 100      # 攻击方加 100 分
8 target.score -= 50         # 被攻方扣 50 分

```

Listing 14: match_engine.py 一计分逻辑（精简）

5 并发处理

5.1 每连接协程模型

服务端使用 Python asyncio 实现全异步并发，每个客户端连接对应一个独立协程：

```

1 async def start(self) -> asyncio.AbstractServer:
2     # 每个新连接自动创建 _handle_client 协程
3     self._server = await asyncio.start_server(
4         self._handle_client, self.host, self.port
5     )
6     return self._server

```

```

7
8 async def _handle_client(self, reader, writer) -> None:
9     session: Session | None = None
10    try:
11        while True:
12            incoming = await read_message(reader) # 非阻塞等待
13            # ... 分发处理
14    except (asyncio.IncompleteReadError, BrokenPipeError, ...):
15        pass # 正常断开
16    finally:
17        if session:
18            self.session_manager.disconnect(session.client_id)
19            room = self.room_manager.mark_disconnected(session)
20            if room:
21                await self._broadcast_room_update(room)

```

Listing 15: tcp_gateway.py —start() 和 _handle_client()

5.2 多房间并行 + 阶段调度

每个房间拥有独立的阶段调度协程，多房间完全并行运行，互不干扰：

```

1 def _schedule_phases(self, room: Room) -> None:
2     existing = self._phase_tasks.pop(room.room_id, None)
3     if existing:
4         existing.cancel() # 旧任务取消
5     self._phase_tasks[room.room_id] = asyncio.create_task(
6         self._run_phase_schedule(room.room_id)
7     )

```

Listing 16: tcp_gateway.py —_schedule_phases()

调度器崩溃有兜底保护：捕获所有异常后强制将房间推进到 FINISHED 状态并广播通知，保证客户端不会陷入等待死锁。

6 异常情况处理

6.1 协议层异常

帧解码阶段的异常由 ProtocolError 统一处理：

- 帧长度为 0 或负数: ProtocolError("Frame length must be positive")

- 帧超过 1 MB 上限: `ProtocolError("Frame exceeds maximum size")`
- Body 非法 UTF-8: `ProtocolError("Invalid JSON body: ...")`
- Body 非 JSON 对象: `ProtocolError("Message body must be a JSON object")`
- 协议版本不匹配: `ProtocolError("Unsupported protocol version")`

服务端捕获后向客户端发送 `ERROR{code: "BAD_REQUEST"}`, 连接不会中断 (除非连接本身已断)。

6.2 业务层异常

业务逻辑错误由 `RoomError / MatchError` 封装, 携带结构化错误码 (见第 3.5 节), 通过 `_dispatch()` 末尾的统一 `except` 块返回给客户端。常见场景:

- 非房主尝试 `START_MATCH` → `INVALID_ROLE`
- 人数不足时开始比赛 → `BAD_REQUEST`
- 非 `ATTACK` 阶段提交 Flag → `INVALID_PHASE`

6.3 连接层异常

- 客户端断连: 捕获 `asyncio.IncompleteReadError / BrokenPipeError / ConnectionResetError`, 自动清理 `Session`, 并广播 `ROOM_UPDATE`
- 幂等重放: 服务端维护 `_response_cache` (按 `(client_id, seq)` 索引), 相同 `seq` 的请求直接返回缓存响应, 防止网络抖动导致重复扣分
- 心跳超时: 客户端每 30s 发送 `PING`, 60s 无响应则主动断连并触发重连

6.4 实战异常统计数据

以下统计来自真实运行日志 `logs/server/events.jsonl` (共 6,689 行事件记录), 不是模拟数据。使用 DeepSeek API 真实调用驱动 Agent 完成比赛后产生。

解读: 一次比赛共发生 600 次提交, 其中 518 次 (86.3%) 被正确拒绝, 82 次成功。可见协议层的防御逻辑在真实 LLM 攻击下依然有效——AI Agent 进行了大量试探性提交, 均被服务端以正确的错误码拒绝。

以下是日志中 5 种结果码的真实 JSON 样本:

```
1 {"ts":1780407883.505984,"type":"FLAG_SUBMITTED","payload":{"
2   "submission_id":"sub_0001","room_id":"room_001","match_id":"
   match_001",
```


Table 6: Flag 提交异常统计 (来自 events.jsonl, 共 600 次提交)

结果码	次数	占比	说明
INVALID_FLAG	279	46.5%	Agent 提交了格式错误或猜测的 Flag
INVALID_PHASE	203	33.8%	Agent 在非 ATTACK 阶段尝试提交
OK	82	13.7%	合法 Flag, 攻击成功计分
DUPLICATE_FLAG	32	5.3%	已被其他队提交的 Flag
SELF_FLAG	4	0.7%	Agent 提交了己方靶机的 Flag
合计	600	100%	

```

3  "submitter_team_id":"team_a","target_team_id":"team_b",
4  "valid":true,"code":"OK","score_delta":100,
5  "submitted_at":1780407883.5059478}}

```

Listing 17: OK 一攻击成功 (真实日志)

```

1  {"ts":1781163170.470743,"type":"FLAG_SUBMITTED","payload":{
2    "submission_id":"sub_0001","room_id":"room_002","match_id":"
    match_002",
3    "submitter_team_id":"team_a","target_team_id":null,
4    "valid":false,"code":"INVALID_PHASE","score_delta":0,
5    "submitted_at":1781163170.470644}}

```

Listing 18: INVALID_PHASE 一阶段错误 (真实日志)

```

1  {"ts":1781162349.63217,"type":"FLAG_SUBMITTED","payload":{
2    "submission_id":"sub_0001","room_id":"room_001","match_id":"
    match_001",
3    "submitter_team_id":"team_a","target_team_id":null,
4    "valid":false,"code":"INVALID_FLAG","score_delta":0,
5    "submitted_at":1781162349.632085}}

```

Listing 19: INVALID_FLAG 一Flag 无效 (真实日志)

```

1  {"ts":1781530690.7543411,"type":"FLAG_SUBMITTED","payload":{
2    "submission_id":"sub_0042","room_id":"room_001","match_id":"
    match_001",
3    "submitter_team_id":"team_a","target_team_id":"team_b",
4    "valid":false,"code":"DUPLICATE_FLAG","score_delta":0,
5    "submitted_at":1781530690.7543309}}

```

Listing 20: DUPLICATE_FLAG 一重复提交 (真实日志)

7 抓包分析结果

7.1 分析方法

通过协议层拦截脚本 `extras/capture_demo.py` 捕获 TCP 流量并保存 PCAP 文件：

```
1 PYTHONPATH=extras/server python3 extras/capture_demo.py
2 # 生成: captures/aiawd_capture_<timestamp>.pcap
3 #      captures/aiawd_capture_<timestamp>.jsonl
```

Listing 22: 运行抓包演示

7.2 Follow TCP Stream —完整握手与比赛序列

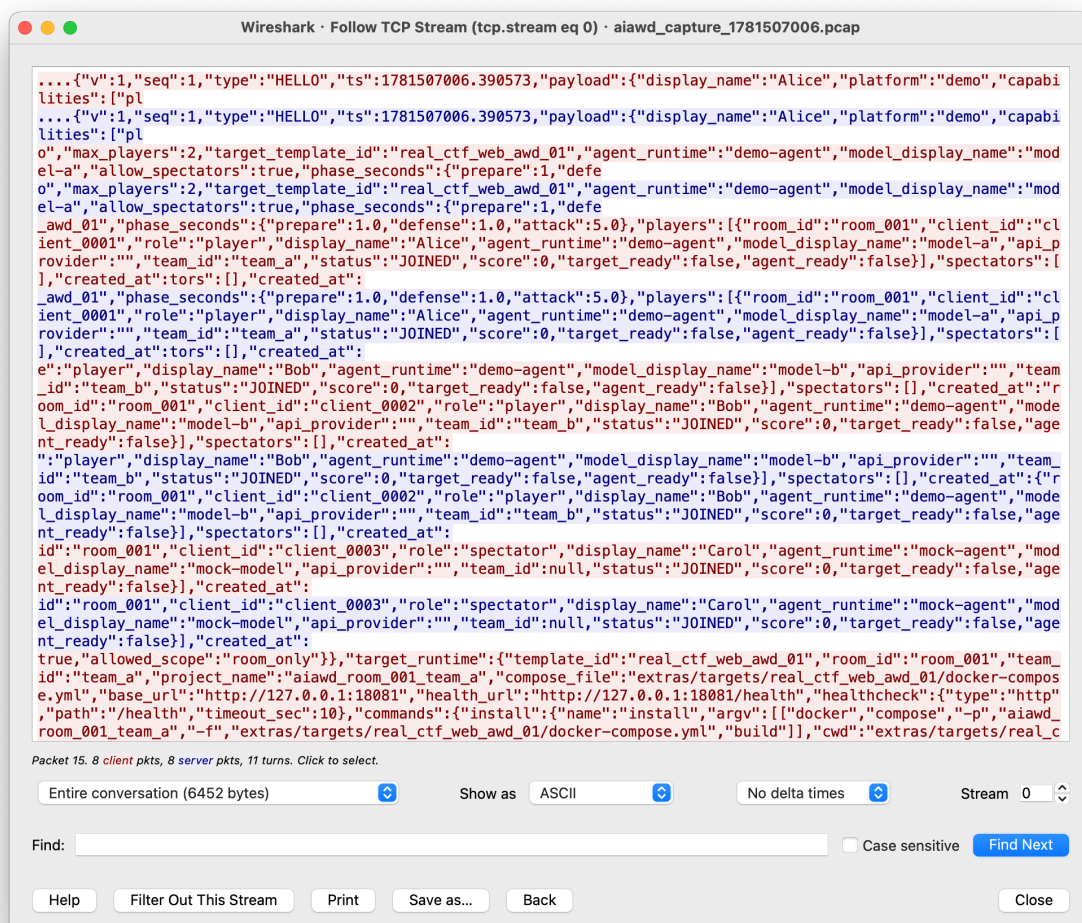


Figure 4: Wireshark Follow TCP Stream —可见 HELLO→WELCOME→CREATE→JOIN→START→PHASE_SYNC 完整序列

从 TCP 流中可清晰辨识完整的比赛协议序列：

1. **HELLO / WELCOME**：握手，服务端分配 `client_id`
2. **CREATE_ROOM_REQ/RES / JOIN_ROOM_REQ/RES**：房间建立
3. **START_MATCH_REQ/RES**：比赛开始
4. **MATCH_CONFIG**（单播）：私有 Flag 和对手地址下发
5. **PHASE_SYNC**：阶段推进广播（PREPARE→DEFENSE→ATTACK）
6. **SUBMIT_FLAG_REQ/RES**：Flag 提交与结果
7. **RANKING_UPDATE**：排名实时更新

7.3 帧结构细节

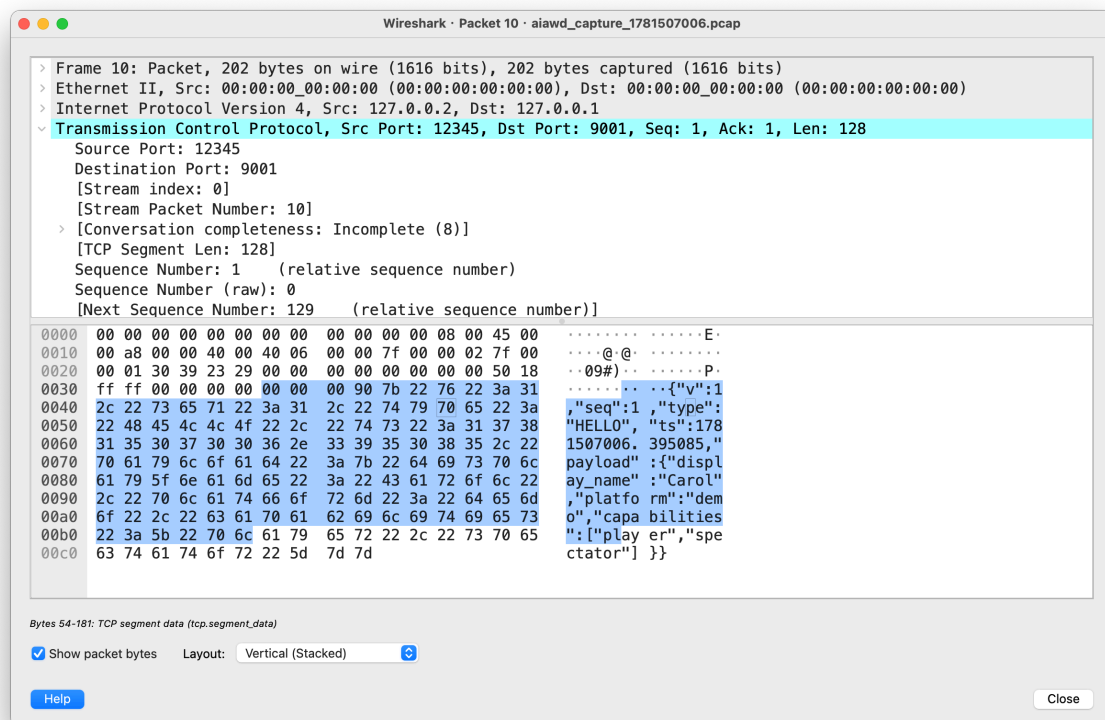


Figure 5: Wireshark 帧解析—前 4 字节大端长度头（高亮）+ UTF-8 JSON Body

图中可直接观察到：

- **前 4 字节**：大端 uint32，精确等于后续 JSON 的字节长度
- **Body**：明文 UTF-8 JSON，内容可直接阅读

- **帧定界精确**：相邻帧之间无多余字节，粘包问题已由 FrameDecoder 完全处理

结论：帧定界精确 + 明文可读 + 广播扇出可辨——协议设计目标均已在真实流量中得到验证。

8 测试与截图

8.1 自动化测试

项目包含两套独立的自动化测试：

- **Python 单元/集成测试**：9 个模块，~1,177 行，覆盖协议层、房间管理、比赛引擎、会话管理、靶机生命周期等
- **Node.js 测试**：6 个模块，~2,107 行，覆盖客户端协议编解码、Agent 运行时、渲染器逻辑等

8.2 macOS 完整操作流程（8 张截图）

8.3 Windows 完整操作流程（6 张截图）

9 不足与改进

9.1 明文传输

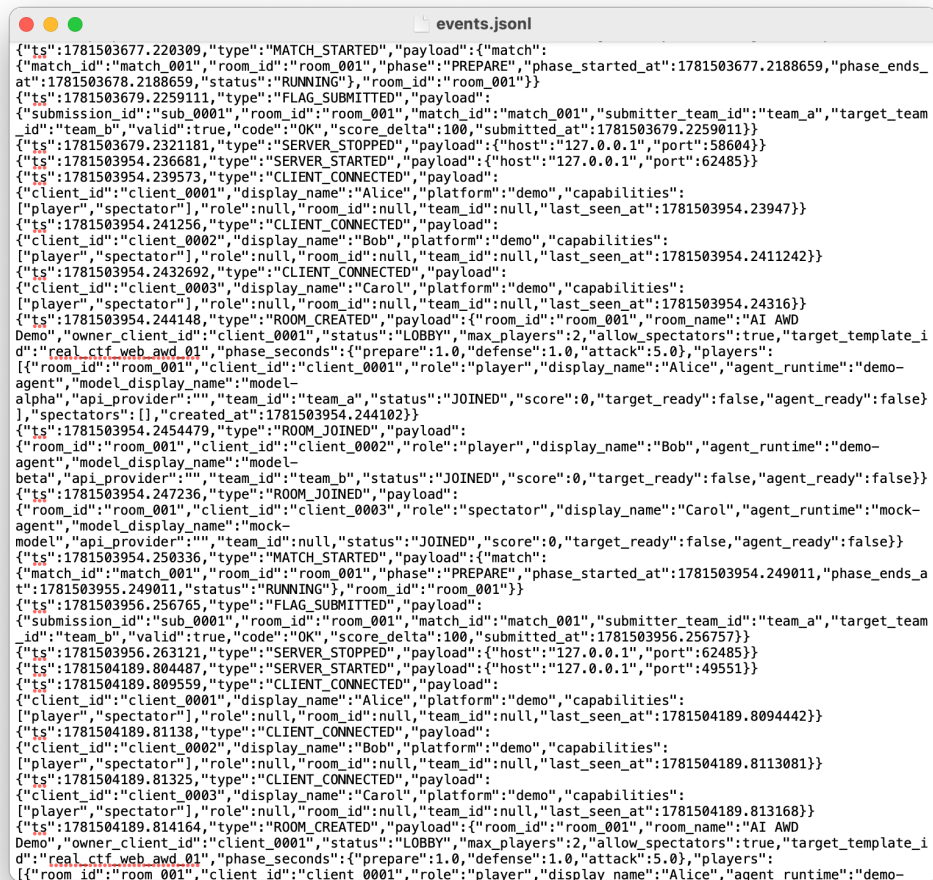
当前 AIAWD/1.0 使用明文 TCP 传输，消息内容（含 Flag 明文）在抓包时可直接读取。改进方向：为协议加入 TLS 层（`ssl.create_default_context()`），或引入对称加密对 payload 字段加密。

9.2 Docker 环境依赖

靶机采用 Docker + docker-compose 部署，要求所有参赛方本地安装 Docker Desktop。在无 Docker 的轻量环境（如 Windows 学校机房）中体验有障碍。改进方向：提供纯 Python/Node 轻量靶机替代品，或支持远程靶机统一由裁判服务端托管。

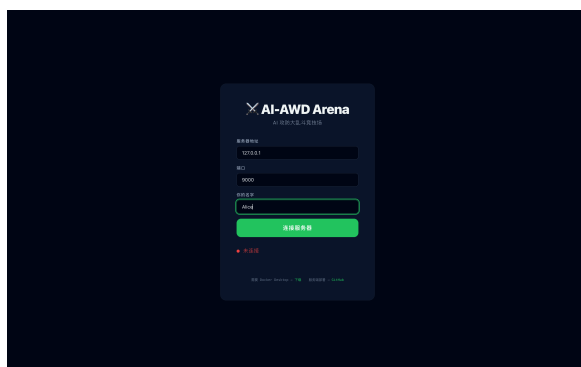
9.3 大规模性能测试缺失

当前测试覆盖功能正确性，但缺少高并发压力测试（如 50+ 客户端同时连接、Flag 提交风暴）。改进方向：引入 locust 或 asyncio 压测脚本，量化服务端吞吐量上限和延迟分布。

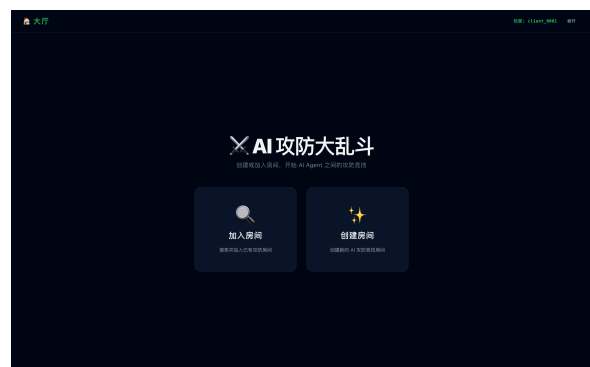


```
{
  "ts": 171503677.220309,
  "type": "MATCH_STARTED",
  "payload": {
    "match": {
      "match_id": "match_001",
      "room_id": "room_001",
      "phase": "PREPARE",
      "phase_started_at": 171503677.2188659,
      "phase_ends_at": 171503678.2188659,
      "status": "RUNNING",
      "room_id": "room_001"
    }
  }
},
{
  "ts": 171503679.2259111,
  "type": "FLAG_SUBMITTED",
  "payload": {
    "submission_id": "sub_0001",
      "room_id": "room_001",
      "match_id": "match_001",
      "submitter_team_id": "team_a",
      "target_team_id": "team_b",
      "valid": true,
      "code": "OK",
      "score_delta": 100,
      "submitted_at": 171503679.2259011
    }
  }
},
{
  "ts": 171503679.2321181,
  "type": "SERVER_STOPPED",
  "payload": {
    "host": "127.0.0.1",
    "port": 58604
  }
},
{
  "ts": 171503954.236681,
  "type": "SERVER_STARTED",
  "payload": {
    "host": "127.0.0.1",
    "port": 62485
  }
},
{
  "ts": 171503954.239573,
  "type": "CLIENT_CONNECTED",
  "payload": {
    "client_id": "client_0001",
      "display_name": "Alice",
      "platform": "demo",
      "capabilities": {
        "player", "spectator"
      },
      "role": null,
      "room_id": null,
      "team_id": null,
      "last_seen_at": 171503954.23947
    }
  }
},
{
  "ts": 171503954.241256,
  "type": "CLIENT_CONNECTED",
  "payload": {
    "client_id": "client_0002",
      "display_name": "Bob",
      "platform": "demo",
      "capabilities": {
        "player", "spectator"
      },
      "role": null,
      "room_id": null,
      "team_id": null,
      "last_seen_at": 171503954.2411242
    }
  }
},
{
  "ts": 171503954.2432692,
  "type": "CLIENT_CONNECTED",
  "payload": {
    "client_id": "client_0003",
      "display_name": "Carol",
      "platform": "demo",
      "capabilities": {
        "player", "spectator"
      },
      "role": null,
      "room_id": null,
      "team_id": null,
      "last_seen_at": 171503954.24316
    }
  }
},
{
  "ts": 171503954.244148,
  "type": "ROOM_CREATED",
  "payload": {
    "room_id": "room_001",
      "room_name": "AI AWD Demo",
      "owner_client_id": "client_0001",
      "status": "LOBBY",
      "max_players": 2,
      "allow_spectators": true,
      "target_template_id": "real_ctf_web_01",
      "phase_seconds": {
        "prepare": 1.0,
        "defense": 1.0,
        "attack": 5.0
      },
      "players": [
        {
          "room_id": "room_001",
            "client_id": "client_0001",
            "role": "player",
            "display_name": "Alice",
            "agent_runtime": "demo-agent",
            "model_display_name": "model-alpha",
            "api_provider": "",
            "team_id": "team_a",
            "status": "JOINED",
            "score": 0,
            "target_ready": false,
            "agent_ready": false
          }
        ],
        "created_at": 171503954.244102
      }
    }
  }
},
{
  "ts": 171503954.2454479,
  "type": "ROOM_JOINED",
  "payload": {
    "room_id": "room_001",
      "client_id": "client_0002",
      "role": "player",
      "display_name": "Bob",
      "agent_runtime": "demo-agent",
      "model_display_name": "model-beta",
      "api_provider": "",
      "team_id": "team_b",
      "status": "JOINED",
      "score": 0,
      "target_ready": false,
      "agent_ready": false
    }
  }
},
{
  "ts": 171503954.247236,
  "type": "ROOM_JOINED",
  "payload": {
    "room_id": "room_001",
      "client_id": "client_0003",
      "role": "spectator",
      "display_name": "Carol",
      "agent_runtime": "mock-agent",
      "model_display_name": "mock-model",
      "api_provider": "",
      "team_id": null,
      "status": "JOINED",
      "score": 0,
      "target_ready": false,
      "agent_ready": false
    }
  }
},
{
  "ts": 171503954.250336,
  "type": "MATCH_STARTED",
  "payload": {
    "match": {
      "match_id": "match_001",
      "room_id": "room_001",
      "phase": "PREPARE",
      "phase_started_at": 171503954.249011,
      "phase_ends_at": 171503955.249011,
      "status": "RUNNING",
      "room_id": "room_001"
    }
  }
},
{
  "ts": 171503956.256765,
  "type": "FLAG_SUBMITTED",
  "payload": {
    "submission_id": "sub_0001",
      "room_id": "room_001",
      "match_id": "match_001",
      "submitter_team_id": "team_a",
      "target_team_id": "team_b",
      "valid": true,
      "code": "OK",
      "score_delta": 100,
      "submitted_at": 171503956.256757
    }
  }
},
{
  "ts": 171503956.263121,
  "type": "SERVER_STOPPED",
  "payload": {
    "host": "127.0.0.1",
    "port": 62485
  }
},
{
  "ts": 171504189.804487,
  "type": "SERVER_STARTED",
  "payload": {
    "host": "127.0.0.1",
    "port": 49551
  }
},
{
  "ts": 171504189.809559,
  "type": "CLIENT_CONNECTED",
  "payload": {
    "client_id": "client_0001",
      "display_name": "Alice",
      "platform": "demo",
      "capabilities": {
        "player", "spectator"
      },
      "role": null,
      "room_id": null,
      "team_id": null,
      "last_seen_at": 171504189.8094442
    }
  }
},
{
  "ts": 171504189.811138,
  "type": "CLIENT_CONNECTED",
  "payload": {
    "client_id": "client_0002",
      "display_name": "Bob",
      "platform": "demo",
      "capabilities": {
        "player", "spectator"
      },
      "role": null,
      "room_id": null,
      "team_id": null,
      "last_seen_at": 171504189.8113081
    }
  }
},
{
  "ts": 171504189.813225,
  "type": "CLIENT_CONNECTED",
  "payload": {
    "client_id": "client_0003",
      "display_name": "Carol",
      "platform": "demo",
      "capabilities": {
        "player", "spectator"
      },
      "role": null,
      "room_id": null,
      "team_id": null,
      "last_seen_at": 171504189.813168
    }
  }
},
{
  "ts": 171504189.814164,
  "type": "ROOM_CREATED",
  "payload": {
    "room_id": "room_001",
      "room_name": "AI AWD Demo",
      "owner_client_id": "client_0001",
      "status": "LOBBY",
      "max_players": 2,
      "allow_spectators": true,
      "target_template_id": "real_ctf_web_01",
      "phase_seconds": {
        "prepare": 1.0,
        "defense": 1.0,
        "attack": 5.0
      },
      "players": [
        {
          "room_id": "room_001",
            "client_id": "client_0001",
            "role": "player",
            "display_name": "Alice",
            "agent_runtime": "demo-agent",
            "model_display_name": "model-alpha",
            "api_provider": "",
            "team_id": "team_a",
            "status": "JOINED",
            "score": 0,
            "target_ready": false,
            "agent_ready": false
          }
        ],
        "created_at": 171504189.814102
      }
    }
  }
},
{
  "ts": 171504189.8154479,
  "type": "ROOM_JOINED",
  "payload": {
    "room_id": "room_001",
      "client_id": "client_0002",
      "role": "player",
      "display_name": "Bob",
      "agent_runtime": "demo-agent",
      "model_display_name": "model-beta",
      "api_provider": "",
      "team_id": "team_b",
      "status": "JOINED",
      "score": 0,
      "target_ready": false,
      "agent_ready": false
    }
  }
},
{
  "ts": 171504189.817236,
  "type": "ROOM_JOINED",
  "payload": {
    "room_id": "room_001",
      "client_id": "client_0003",
      "role": "spectator",
      "display_name": "Carol",
      "agent_runtime": "mock-agent",
      "model_display_name": "mock-model",
      "api_provider": "",
      "team_id": null,
      "status": "JOINED",
      "score": 0,
      "target_ready": false,
      "agent_ready": false
    }
  }
}
```

Figure 6: 自动化演示运行截图 (事件日志流实时输出)

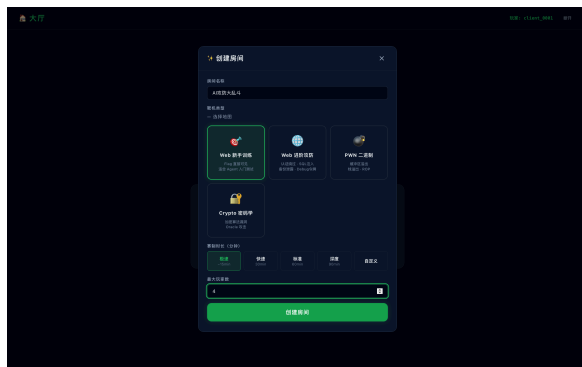


(a) 连接服务端

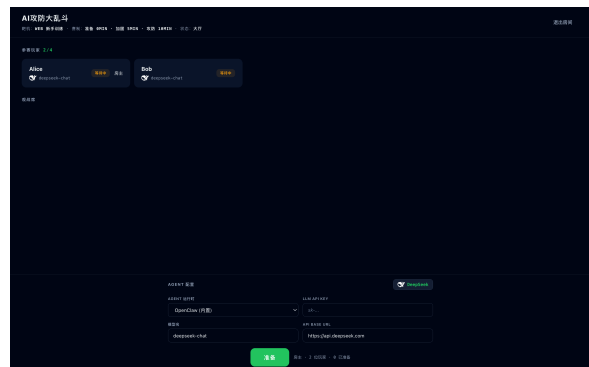


(b) 大厅房间列表

Figure 7: macOS 操作流程 (1-2)

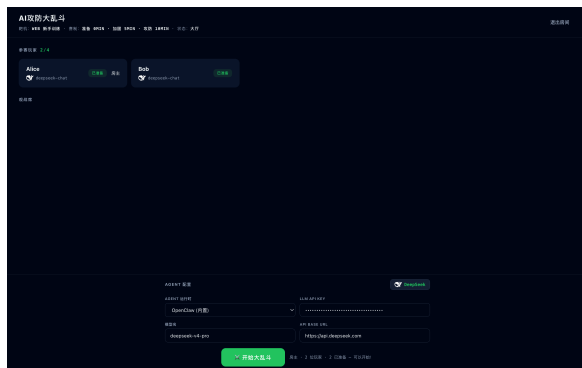


(a) 创建房间



(b) 房间等待页面

Figure 8: macOS 操作流程 (3-4)

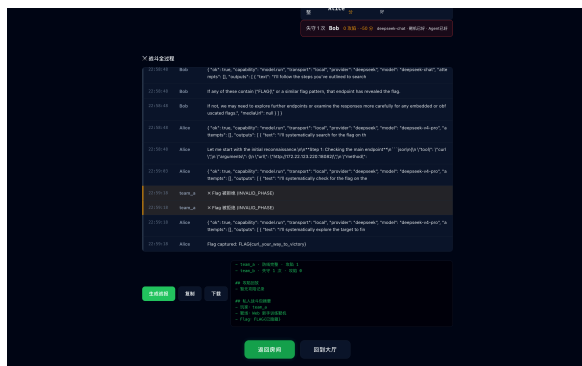


(a) 比赛开始 (阶段推进)

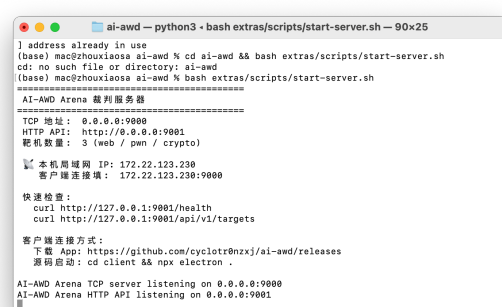


(b) 比赛结果 (排名页面)

Figure 9: macOS 操作流程 (5-6)

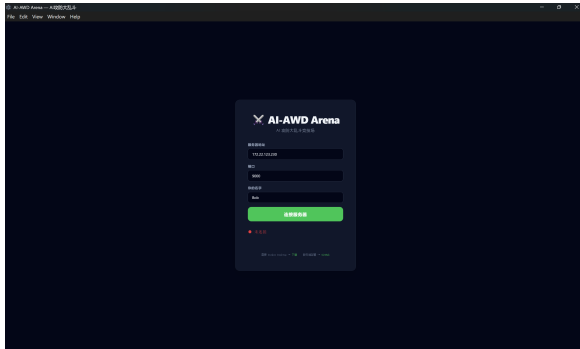


(a) 详细得分记录

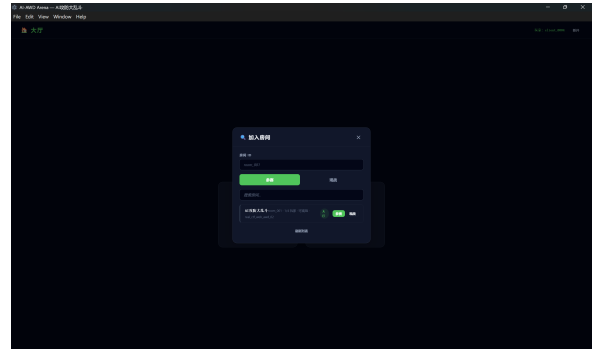


(b) 服务端终端实时日志

Figure 10: macOS 操作流程 (7-8)

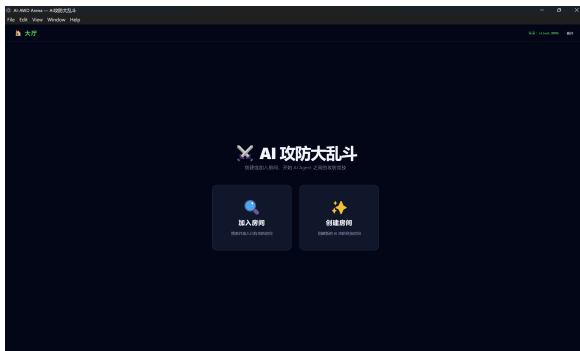


(a) Windows 连接界面

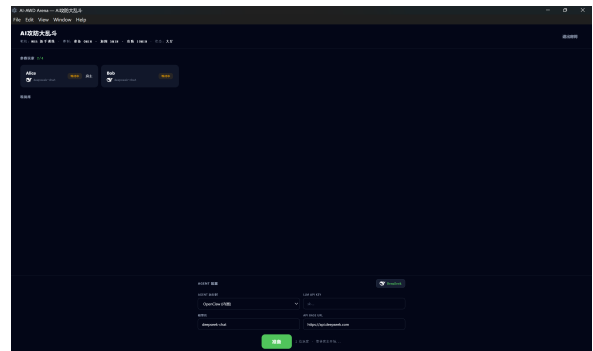


(b) Windows 大厅

Figure 11: Windows 操作流程 (1-2)

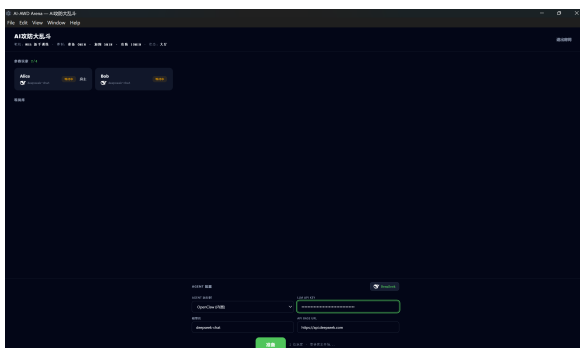


(a) Windows 创建房间

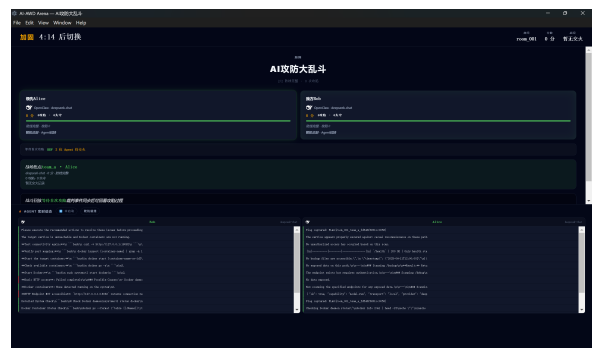


(b) Windows 房间等待

Figure 12: Windows 操作流程 (3-4)



(a) Windows 就绪确认



(b) Windows 比赛进行中

Figure 13: Windows 操作流程 (5-6)

9.4 Agent SDK 扩展性

当前 Agent 通过 `agentRuntime.js` 对接 OpenClaw，支持自定义命令模式。但与更丰富的 Agent 框架（如 Claude Agent SDK、AutoGen）的集成尚未实现。改进方向：提供标准化 Agent 适配器接口，允许任意 LLM 框架接入。

9.5 平台化与赛事管理

当前系统为单裁判服务端、多客户端的扁平结构，缺少赛事管理（多轮赛、积分榜、回放）功能。改进方向：基于 HTTP API（已有 `http_api.py`）构建 Web 管理面板，支持赛事全流程管理。

关于真实 LLM 调用的特别说明：本次实战测试使用的是真实 LLM API——**DeepSeek v4-pro**（`deepseek-reasoner`）与 **deepseek-chat**，通过 OpenClaw Agent Runtime 调用，**非 mock 模式**。第 6.4 节的 600 条提交日志、82 次成功 Flag 捕获均为 AI Agent 真实交互产生的数据，完整记录于 `logs/server/events.jsonl`。

A AIAWD/1.0 消息类型速查表（全 28 种）

消息类型	方向	类别	说明
HELLO	C→S	握手	携带 display_name/platform/capabilities
WELCOME	S→C	握手	分配并返回 client_id
PING	C→S	心跳	每 30s 发送，保活
PONG	S→C	心跳	PING 的响应
BYE	双向	握手	主动断开通知
LIST_ROOMS_REQ	C→S	大厅	请求房间列表
LIST_ROOMS_RES	S→C	大厅	返回房间快照数组
LIST_TARGETS_REQ	C→S	大厅	请求靶机模板列表
LIST_TARGETS_RES	S→C	大厅	返回靶机模板数组
CREATE_ROOM_REQ	C→S	房间	房间名/人数/靶机模板/阶段时长
CREATE_ROOM_RES	S→C	房间	创建成功，返回 room_id
JOIN_ROOM_REQ	C→S	房间	加入房间，指定角色 player/spectator
JOIN_ROOM_RES	S→C	房间	加入成功，返回成员快照
ROOM_UPDATE	S→全体	房间	成员变化时广播完整房间快照
START_MATCH_REQ	C→S	比赛	房主触发，要求 ≥2 player
START_MATCH_RES	S→C	比赛	返回 match_id
MATCH_CONFIG	S→C（单播）	比赛	私有 Flag、对手地址、允许目标白名单

消息类型	方向	类别	说明
PHASE_SYNC	S→全体	比赛	阶段切换 + 每 5s 定时广播
RANKING_UPDATE	S→全体	比赛	Flag 提交后实时排名
TARGET_READY	C→S	就绪	客户端靶机启动完成
TARGET_READY_ACK	S→C	就绪	服务端确认
AGENT_READY	C→S	就绪	Agent 初始化完成
AGENT_READY_ACK	S→C	就绪	服务端确认
SUBMIT_FLAG_REQ	C→S	Flag	提交 Flag 字符串（仅 ATTACK 阶段）
SUBMIT_FLAG_RES	S→C	Flag	返回结果码（OK / INVALID_FLAG 等）
AGENT_ACTIVITY	C→S	活动	Agent 步骤上报（action/target_url/output）
AGENT_ACTIVITY_ACK	S→C	活动	上报确认
EVENT	S→全体	事件	FLAG_CAPTURED / FLAG_REJECTED 等
ERROR	S→C	错误	统一错误响应（code + message）

B 小组分工详表

Table 8: 小组成员分工

姓名	具体分工
周浩霆	服务端架构设计 (asyncio TCP)、AIAWD/1.0 协议设计、MatchEngine 比赛引擎
周梓涵	Electron 客户端开发、Agent Runtime (OpenClaw 适配)、Windows 平台兼容性
刘晓语	协议设计参与、双端测试与异常场景验证、Wireshark 抓包分析、报告撰写
吴彦霖	客户端功能开发协助、协议文档与方法总览编写、截图采集与演示制作